1 Lesson 04-11-26

1.1 CAN

CAN (*Controller Area Network*) is a protocol aimed to increase communication rate, and increase interconnect length.

Defined by Bosch in 1980, it supports a *multi-node* topology, with multiple blocks on the same bus, and all blocks acting as masters (one-to-many with node arbitration).

It was developed for automotive applications, but also used in other contexts. It's designed for noisy environments.

The protocols defines several aspects of the communication:

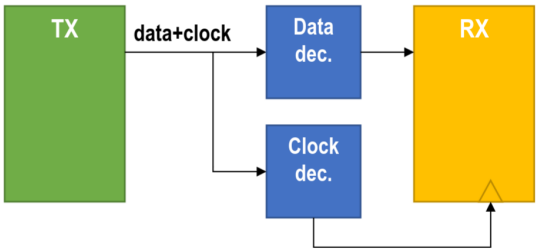
- **Physical** (electrical) level
- **Transaction** (frame format) level
- **Software** (application) level

It is a serial communication with an embedded clock, which means it has a high clock frequency, not affected by skew (though it isn't suited to short interconnections).

Of course, it requires special encodings and circuits, such as clock decoders at the receiver, ro extract the clock information and the data stream.

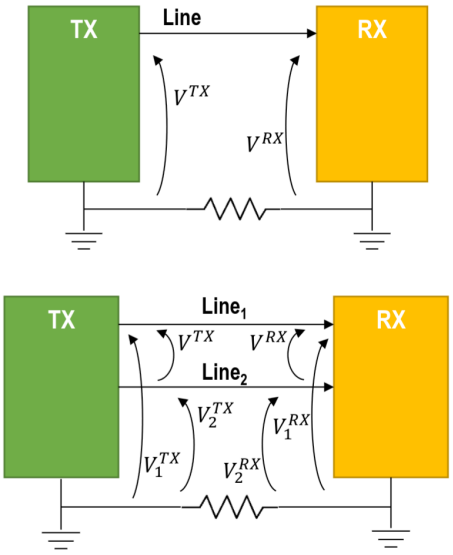
As for increasing interconnection length, skew is very high for long interconnections, so we use a serial communication with embedded clock.

We also use *differential signaling* to reduce the effects of noise.



1.1.1 Differential signaling

Now, in a real communication system, if we transmit signals via 2 wires (data and ground), we never get exactly the same voltage at RX and TX $V^{TX} \neq V^{RX}$. This is because of ground differences, making the ground not a good reference for the data line (which suffers from interference of its own).



What we do is encode information as the voltage difference between two wires. Assuming V_{CM} is equal to the difference between the two grounds, and interference carried on both the data wires (*Common Mode Noise*, **CMD**), we have:

$$V_1^{RX} = V_1^{TX} + V_{CM}, \quad V_2^{RX} = V_2^{\text{ő}X} + V_{CM}$$

And we can recover, via operational amplifiers at both ends:

$$V^{RX} = V_1^{RX} - V_2^{RX} = V_1^{TX} + V_{CM} - V_2^{TX} - V_{CM} = V_1^{TX} - V_2^{TX} = V^{TX}$$

This both RX and TX see the same voltage, whatever the ground differences, losses, and other interference.

Let's focus on what we mean by interference: any other unwanted interference that equally affects both wires induces *common mode noise*, which is cancelled by differential signaling.

This means we don't want wires affected by different noise (*differential mode nosie*), that is noise source close to one wire, for instance.

To aid in this, we *twist* the pair of wires so that the interference alternatively affects one wire and the other. Assuming the interference is uniform along multiple twists, the noise becomes common mode, and is cancelled by differential signaling. We can also *shield* the pair to reject interferences.

This gives a reason behind the wires we are used to:

- **STP** (*Shielded Twisted Pair*);
- **UTP** (*Unshielded Twisted Pair*).

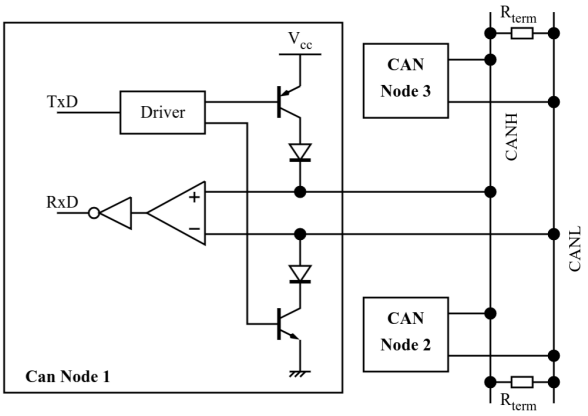


So, we have that the CAN protocol does differential signaling on two wires: **CANH** and **CANL**.

The two wires are used for data transfer, not one for data and the other for clock, like in I2C: there is no dedicated wire for the clock, so we don't suffer the problem of skew.

Data is in fact encoded as the difference CANH - CANL, which allows for long interconnections through embedded clock encoding. The clock is recovered from data.

Let's look at the CAN bus itself.

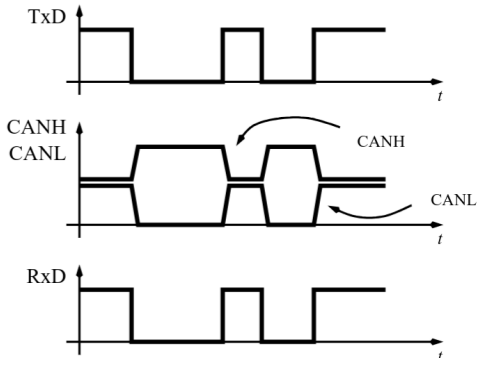


We notice open collector driving of CANH and CANL, driving on the high side (through a PNP) for CANH, and on the low side (through an NPN) for CANL. Reading, as we said, is done through a differential amplifier $V_{out} = CANH - CANL$, with an invertd output.

We have a terminating resistances between CANH and CANL, with value $120\,\Omega$, meaning the resistance between the two CAN lines is $60\,\Omega$. This resistance is used to equalize the two wires voltage values when floating, and used to avoid reflections on the transmission line, especially for long interconnections.

If each side of the can bus can only be driven in one direction (through open collector signaling), we wonder how logic levels are represented. To solve this, we use a system that distinguishes floating (*recessive*) values from driven ones:

- **Logic 0** (*dominant*): $CANH = 5\,\text{V}$, $CANL = 0\,\text{V}$, difference in reading is high, output of the input amplifier is low;
- **Logic 1** (*recessive*): $CANH = HiZ$, $CANL = HiZ$, difference in reading is 0 (because of the equalizing resistance), output of the input amplifier is high.



Let's talk about bus contention: all nodes have the same interface on the CAN bus. When not transmitting, each node generates a recessive bit, so a high value on receiver,

and CANH and CANL are floating. A dominant bit during transmission always wins over a recessive bit, as it means some node is driving the lines. Since each node drives the lines to the same values on a dominant bit (open collector driving), in case of bus contention there are no damages or short circuits.

Since we said that NRZ encoding is used, with no guaranteed transitions we do bit stuffing after 5 consecutive identical bits:

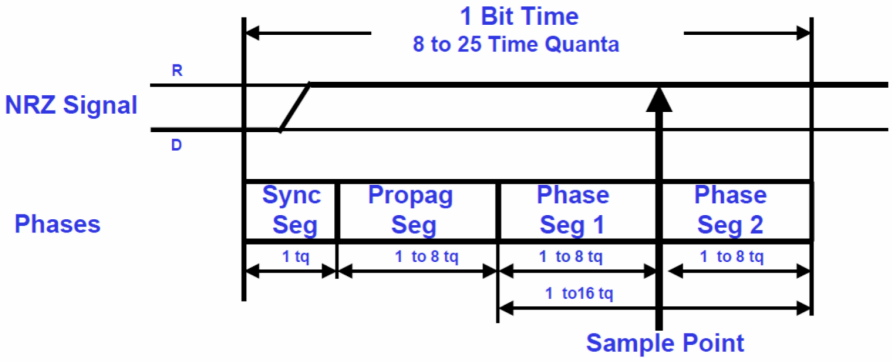
- After 5 dominant (0), insert a recessive (1);
- After 5 recessive (1), insert a dominant (0).

If 5 consecutive identical bits are followed by the opposite value, bit stuffing is applied anyway, so that the receiver can delete stuffed bits when decoding the incoming bitstream.

The eye diagram sees the bit time divided into 4 time quanta (8 to 25):

- Synch segment
- Propagation segment
- Phase segment 1
- Phase segment 2

Sampling is specified to happen between Phase1 and Phase2 segments.

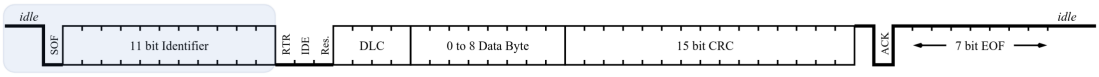


1.1.2 I2C protocol

Enough electronics. Let's talk about the protocol.

Three main problems:

- **Synchronization**: all devices have to be synchronized at each recessive to dominant transition, with transitions assumed to happen at the beginning of a bit time. Each device momentarily adjusts Phase1 and Phase2 lengths to synch. All devices start in low speed, and speed can be modified later at run-time;
- **Addressing**: when transmitting, each node generates an 11 bit address, known as identifier in CAN. It is *not* the address of the destination node;
- **Arbitration**: similar to I2C, the lower identifier wins contention (dominant vs. recessive), as lower identifier means higher priority.



Let's see the full transmission in pratice.

1. At idle, we see a recessive bit on the bus;
2. *Start Of Frame* (**SOF**) is a dominant bit generated by a node. It guarantees a recessive to dominant transition from the idle state. All other nodes synchronize their own clock, and avoid transmitting until the end of the frame;
3. Then the *identifier* is transmitted, on 11 bits (possibly extended to 29 through IDE);
4. The *Remote Transmission Request* (**RTR**) follows.

   If a node wants to transmit data to other nodes, it is dominant to signal for data frames.

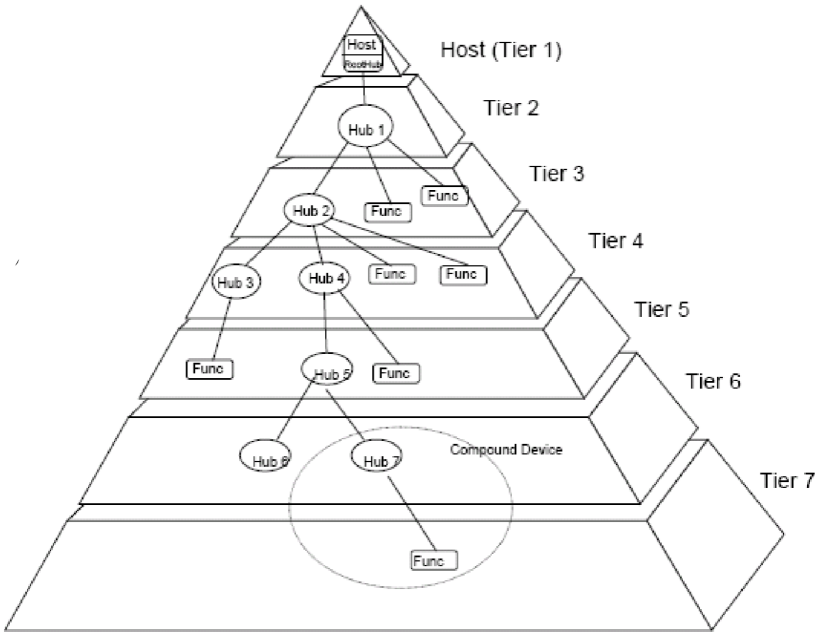
   If a node requires data from another node it is recessive and the DLC indicates the number of requested bytes of data. Of course in this last case there is no data field.
5. As we said, there's an **IDE** (*ID Extension*) mode, flagged by the IDE bit (dominant for 11 bits);
6. A reserved dominant bit is included in this flagging section;
7. The *Data Length Code* (**DLC**) follows, including the number of subsequent data bytes. It's a field on 4 bits, with acceptable values 0 to 8;
8. We can finally transmit data. The *data* field is 0 to 8 bytes (depends on DLC, 0 to 64 bits), but always a multiple of 8 bits;
9. A *Cyclic Redundancy Code* (**CRC**) is performed on 15 bits. One recessive bit flags the end of it;
10. There's an acknowledge (**ACK**) bit: transmitter generates recessive, receivers generate dominant. This is also delimited by a recessive bit;
11. Then we have the *End Of Frame* (**EOF**), given by 7 consecutive bits, all recessive. Note that this is an intentional violation of the bit stuffing rule: bit stuffing is otherwise applied to all fields, except CRC delimiter, ACK and EOF.
12. Now the bus returns idle, that is the always recessive. Other nodes can now start a transmission. Good for them.

Notice that CAN defines higher protocol levels. At the electrical level, all nodes on the bus are masters: any node can start a transmission, at any given time. Arbitration on identifiers used to manage priorities and reception masks.

At higher levels of the protocol, some node have additional tasks: for instance to configure the bus and the other nodes, change the communication speed... Just to summarize, several different higher level protocols can be dfined on top of the CAN physical layer.

1.2 USB

We all know **USB** (*Universal Serial Bus*). It was designed to substitute several kind of legacy connections, such as PS/2, RS232, ... It supports Hot Plug&Play, and different speeds (three in USB 2.0).



The USB Network has a tree structure: there's a single master (known as the *host*), and up to 127 slaves (known as *functions* or *devices*), organized on up to 7 levels. Always point-to-point communication between the single master and a slave; *hubs* virtualize connections.

Since the USB Protocol defines a single master, there's no need for arbitration. Each function (FUN) is assigned a unique address between 1 e 127, assigned when it is initially plugged in to the network. Each FUN provides a maximum of 32 (16+16) end-points, with indexes 0 to 15. For each index, there can be an end-point for reading, and another for writing. Each end-point is unidirectional, with end-point 0 being always present

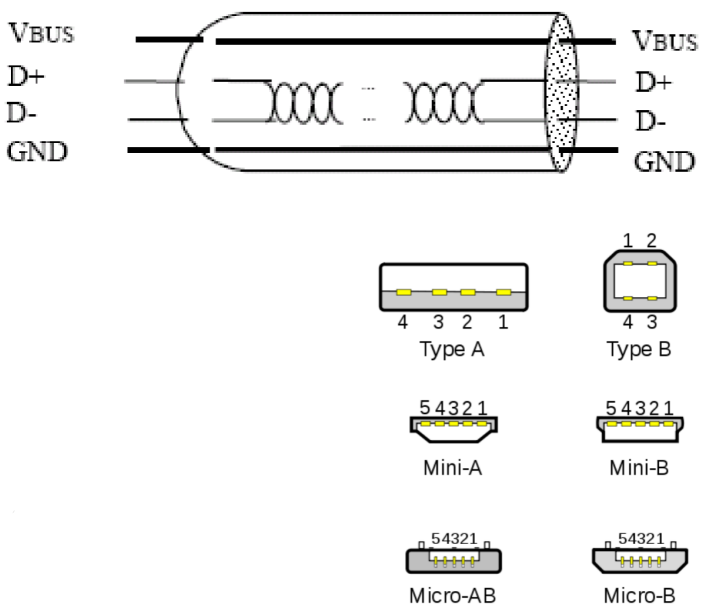
The USB standard is huge. It defines the behavior at several abstraction levels:

- Mechanical
- Electrical
- Protocol
- Transactions
- Multiple transfers
- Driver
- Application and managing software

The *mechanical* standard is based on 4 wires, two for power:

- V_{BUS} at $5\,\mathrm{V}$, $500\,\mathrm{mA}$;
- The ground wire.

And a differential pair for data: D^+ and D^- on STP.



There are several kinds of physical connectors, that can be different between host and function.

USB does full differential signaling, with different voltage levels according to standard and speed configuration. It features error checking and NRZI coding:

- **Logic 0**: transition (at the beginning of a bit);
- **Logic 1**: maintains previous value.

1.2.1 USB pipes

At the *transaction* level, communication on USB based on **pipes** (logical channels). A pipe can be of stream or message type:

- Stream pipe:
  - **Isochronous**: guaranteed bandwidth, possibly loosing data (audio, video, ...);
  - **Interrupt**: quick answer and guaranteed latency (keyboard, mouse, ...);
  - **Bulk**: large sporadic transfers, served using a best-effort bandwidth (file transfers, ...).
- Message pipe:
  - **Control**: short commands to devices and status messages (configuration on endpoint 0, ...).

1.3 Processo peripherals